ODOO HACKATHON 2026 | GREEN & RENEWABLE ENERGY TRACK | VERSION 1.0.0 (RC1) | SINGLE SOURCE OF TRUTH (SSOT)

DustWatch Software Requirements & Architecture Specification

An autonomous financial decision engine & operations platform built on Odoo 19 for solar installation businesses, linking 4 developers, their autonomous AI agents, and Git around a unified implementation contract.

> *"Today owners clean by the calendar. DustWatch lets them clean by the numbers."*

1. SYSTEM VISION & DUAL-DEPLOYMENT PHILOSOPHY

Solar panels in arid Gulf environments face heavy soiling: studies confirm a **12.7% generation loss in 5 months** and up to **27% in peak dust months** (~1%/day). Conventional solar operations either clean too rarely (bleeding electricity revenue) or too often (burning cash and desalinated water). DustWatch solves this through continuous financial optimization.

Mode A: Modular Add-On for Established Installers

For solar contractors with an existing ERP/CRM. DustWatch hooks directly into native Odoo:

- Attaches solar installation sites to existing customer records (`res.partner`).
- Dispatches field work orders through Odoo Field Service / Project tasks (`project.task`).
- Auto-generates verified customer invoices upon job completion (`account.move`).
- Exposes JSON-RPC/REST endpoints for external CRMs (Salesforce, HubSpot, custom).

Mode B: Turnkey All-in-One Operations Platform

For new solar contractors seeking an out-of-the-box system from the ground up:

- Full customer lifecycle: Lead capture → Rooftop site survey → System commission.
- Inverter telemetry ingestion via automated cron or drag-and-drop CSV wizard.
- Visual Operations Kanban board with color-coded health alerts (Green, Amber, Red).
- Customer self-service portal displaying transparent proof of recovery and ROI.

2. AUTONOMOUS HIGH-LEVEL SYSTEM WORKFLOW

1. Telemetry Ingest (Inverter kWh + Sun Hours) → **2. Expected Baseline** (Cap × Sun × PR) → **3. Price Loss (AED)** (Lost kWh × Tariff) → **4. Breakeven Trigger** (Loss ≥ Clean Cost) → **5. Recovery Audit** (Payback & PR Tuning)

3. ODOO 19 MODULE DIRECTORY ARCHITECTURE

The codebase adheres to official Odoo 19 module layout guidelines and is fully version-controlled:

```
dustwatch/
├── __manifest__.py        # Metadata, dependencies (base, contacts), views & security registration
├── models/
│   ├── site.py            # dustwatch.site: Core rooftop solar asset & cumulative loss ledger
│   ├── reading.py         # dustwatch.reading: Daily time-series generation telemetry & loss math
│   ├── job.py             # dustwatch.job: Cleaning work order lifecycle & post-clean recovery audit
│   └── res_partner.py     # res.partner extension: Links solar portfolios & stats to customers
├── views/                 # XML views: site_views.xml, reading_views.xml, job_views.xml, menu_views.xml
├── wizards/               # import_readings_wizard.py: CSV telemetry ingestion modal
├── security/              # security_groups.xml, ir.model.access.csv (RBAC matrix)
└── data/                  # ir_cron_data.xml (midnight trigger), sample_demo_data.xml, 90-day CSV
```

4. DETAILED DATA DICTIONARY & FIELD SPECIFICATIONS

All developers and AI assistants must strictly respect the technical field names, types, and compute bindings below:

4.1 Model: `dustwatch.site` (Solar Rooftop Asset)

Technical Field	Field Type	Attributes	Description & Business Logic
`name`	`Char`	required=True	Installation identifier (e.g. "Grand Hyatt Hotel Main Roof").
`partner_id`	`Many2one`	`res.partner`	Customer / Building Owner who owns the solar asset.
`capacity_kwp`	`Float`	default=100.0	Peak DC power rating in kilowatts-peak (kWp) under standard test conditions.
`electricity_price`	`Float`	default=0.40	Electricity tariff in local currency (AED/kWh) paid by the customer.
`cleaning_cost`	`Float`	default=1500.0	Contractor fee in AED for a single comprehensive panel wash.
`performance_factor`	`Float`	default=0.80	Performance Ratio (PR) baseline; dynamically recalibrated after each clean.
`date_last_cleaned`	`Date`	default=today	Baseline date; only daily readings on/after this date accrue to running loss.
`running_loss_aed`	`Float`	compute, store	$\sum ext{lost_aed}$ since `date_last_cleaned`. The financial trigger counter.
`cleaning_progress`	`Float`	compute, store	$(ext{running_loss_aed} / ext{cleaning_cost}) imes 100$. Percentage towards trigger.
`health_state`	`Selection`	compute, store	`'green'` (<70%), `'amber'` (70%-99%), `'red'` (≥100% - Clean Due!).

4.2 Model: `dustwatch.reading` (Daily Telemetry)

Technical Field	Field Type	Attributes	Description & Business Logic
`site_id`	`Many2one`	`dustwatch.site`	Relational foreign key linking reading to parent solar rooftop.
`date`	`Date`	required=True	Reading timestamp (daily resolution).
`sun_hours`	`Float`	default=5.5	Peak Sun Hours (PSH) measured today from pyranometer or solar weather API.
`actual_kwh`	`Float`	required=True	Physical kilowatt-hours generated, recorded by on-site solar inverters.
`expected_kwh`	`Float`	compute, store	$ ext{capacity_kwp} imes ext{sun_hours} imes ext{performance_factor}$.
`lost_kwh`	`Float`	compute, store	$\max(0, ext{expected_kwh} - ext{actual_kwh})$. Output lost to soiling.
`lost_aed`	`Float`	compute, store	$ ext{lost_kwh} imes ext{electricity_price}$. Financial loss today.
`soiling_loss_pct`	`Float`	compute, store	$(ext{lost_kwh} / ext{expected_kwh}) imes 100$. Soiling degradation percentage.

4.3 Model: `dustwatch.job` (Cleaning Work Order)

Technical Field	Field Type	Attributes	Description & Business Logic
`name`	`Char`	required=True	Auto-generated reference (e.g. `CLEAN/202611/Grand Hyatt`).
`site_id`	`Many2one`	`dustwatch.site`	Rooftop site requiring cleaning dispatch.
`state`	`Selection`	default='proposed'	`'proposed'` → `'scheduled'` → `'done'` → `'cancelled'`.
`trigger_loss_aed`	`Float`	tracked	Accumulated loss that triggered the work order (e.g. AED 1,515).
`cost`	`Float`	required=True	Agreed contractor cleaning charge (defaults to site cleaning cost).
`recovery_kwh_day`	`Float`	compute, store	$\overline{ ext{Loss}}_{ ext{pre 3d}} - \overline{ ext{Loss}}_{ ext{post 3d}}$. Output recovered per day.
`payback_days`	`Float`	compute, store	$ ext{cost} / (ext{recovery_kwh_day} imes ext{tariff})$. Payback duration in days.
`is_fault_flagged`	`Boolean`	compute, store	True if generation fails to recover post-clean, indicating an inverter fault.

5. THE MATHEMATICAL DECISION ENGINE (FORMULAS 6.1 – 6.7)

DustWatch operates on a mathematically deterministic framework that replaces subjective guesswork with rigorous optimization:

6.1 Expected Daily Electricity Output (Clean Baseline)
Theoretical clean generation adjusted for daily measured peak sun hours

```
E = Capacity (kWp) × Sun Hours × PR (0.80)
```

6.2 Electricity Lost to Dust (Daily Soiling Deficit)
Clamps negative values to zero during exceptionally clear atmospheric days

```
Lost kWh = max( 0, Expected kWh - Actual kWh )
```

6.3 Daily Financial Loss
Monetizes physical energy losses into property owner currency

```
Money Lost (AED) = Lost kWh × Electricity Tariff (AED/kWh)
```

6.4 & 6.5 Optimal Cleaning Trigger (Breakeven Rule)
Automated dispatch rule: cumulative loss balance matches contractor invoice

```
Trigger Job IF: Running Loss (AED) ≥ Cost of Clean
```

Analytical Proof of Breakeven Optimality

Let C_c be the fixed cleaning cost, and let daily dust accumulation rate cause a linear daily loss $L(t) = k \cdot t$. Over a cycle of T days, cumulative lost energy equals $\int_0^T k t \, dt = rac{1}{2} k T^2$. The average total cost per day is $f(T) = rac{C_c + rac{1}{2} k T^2}{T} = rac{C_c}{T} + rac{1}{2} k T$. Minimizing by differentiating with respect to T: $f'(T) = - rac{C_c}{T^2} + rac{1}{2} k = 0 \implies rac{1}{2} k T^2 = C_c$. **The total cost of dust is minimized exactly when cumulative lost energy revenue equals the cleaning fee.**

6.6 Recovery & Payback Speed
Audits recovery & calculates payback days

```
Payback Days = Cost / Recovery_AED
```

6.7 Dynamic Baseline Learning
Auto-tunes PR from 3-day post-clean output

```
PR_new = Avg(Actual / (Cap × Sun))
```

6. HANDLING EDGE CASES: WEATHER, RAIN & INVERTER FAULTS

1. Cloud Cover
On overcast days, measured sun hours drop automatically. Because Expected kWh drops proportionally, DustWatch never mistakes clouds for dust.

2. Desert Rain Showers
Rain washes panels for free. Actual generation rebounds immediately, daily loss drops to near zero, and running loss freezes, naturally postponing cleans.

3. Inverter / Hardware Failure
If output remains depressed after a clean, Formula 6.6 detects zero recovery and sets `is_fault_flagged = True`, alerting technicians to an electrical defect.

7. UI/UX & VIEWS SPECIFICATION

DustWatch presents a clean, responsive CRM/Dashboard interface tailored for solar operations managers:

7.1 Solar Sites Portfolio Tree View (`dustwatch.site.tree`)

Uses Odoo conditional row decorators to visually triage installations by urgency:

```
<list string="Solar Sites Portfolio"
      decoration-danger="health_state == 'red'"
      decoration-warning="health_state == 'amber'"
      decoration-success="health_state == 'green'">
    <field name="name" font-weight="bold"/>
    <field name="partner_id"/>
    <field name="capacity_kwp" sum="Total kWp"/>
    <field name="days_since_last_clean"/>
    <field name="running_loss_aed" sum="Total Forfeited AED"/>
    <field name="cleaning_cost"/>
    <field name="cleaning_progress" widget="progressbar"/>
    <field name="health_state" widget="badge"/>
</list>
```

7.2 Operations Kanban Dashboard (`dustwatch.site.kanban`)

Displays solar assets in interactive cards grouped by health status. Red cards display an instant **"Dispatch Clean"** action button that automatically generates a work order without navigating away.

7.3 Cleaning Dispatch Kanban Board (`dustwatch.job.kanban`)

A field service board with three primary workflow stages:

1. Proposed by App
Generated automatically by nightly cron when running loss ≥ cleaning cost.

2. Dispatched / Scheduled
Assigned to field crew; date scheduled and water conservation targets set.

3. Cleaned & Verified
Completed job; triggers baseline reset and post-clean 3-day recovery audit.

7.4 Two-Line Telemetry Graph View (`dustwatch.reading.graph`)

Embedded multi-series line chart plotting `expected_kwh` (green benchmark) against `actual_kwh` (orange curve), providing instant visual proof of gradual dust sag, sandstorm drops, and post-clean jumps.

7.5 Telemetry Ingestion Wizard (`dustwatch.import.readings.wizard`)

Modal dialog accessible from Tools menu. Ingests CSV files with headers: `date,sun_hours,actual_kwh,notes`. Parses records in memory and bulk creates telemetry rows with instant toast notification feedback.

8. SECURITY ARCHITECTURE & ROLE-BASED ACCESS CONTROL (RBAC)

Security rules ensure field technicians, operations managers, and commercial customers have proper data isolation:

User Group	Technical XML Identifier	Access Permissions & Record Scope
Field Technician / Crew	`group_dustwatch_user`	Read access to all sites; Create/Write daily readings; Update job status to 'done'. Cannot delete records or modify tariffs.
Operations Manager / Admin	`group_dustwatch_manager`	Full CRUD on all DustWatch models, system configuration, tariffs, cron schedules, and CSV import wizards.
Solar Client (Portal)	`group_dustwatch_portal`	Read-only access restricted via Odoo record rule to sites where `partner_id = user.partner_id`.

9. NATIVE ODOO ECOSYSTEM INTEGRATIONS

1. CRM / Contacts (`res.partner`)

Extends partner views with smart buttons showing total solar sites, total kWp capacity, and verified lifetime financial savings.

2. Field Service / Projects

`dustwatch.job` hooks into Odoo `project.task`, allowing dispatch of cleaning crews with mobile checklist validation.

3. Accounting & Invoicing

Moving a cleaning job to 'done' can automatically draft an `account.move` customer invoice for the agreed cleaning fee.

10. AUTOMATED NIGHTLY CRON ARCHITECTURE

An automated scheduled action (`ir_cron_dustwatch_evaluate_triggers`) runs nightly at 00:05 UTC:

```
@api.model
def cron_evaluate_cleaning_triggers(self):
    sites_due = self.search([('active', '=', True), ('health_state', '=', 'red')])
    for site in sites_due:
        open_jobs = site.job_ids.filtered(lambda j: j.state in ['proposed', 'scheduled'])
        if not open_jobs:
            self.env['dustwatch.job'].create({
                'site_id': site.id,
                'trigger_loss_aed': site.running_loss_aed,
                'state': 'proposed',
                'cost': site.cleaning_cost,
            })
```

11. AI AGENT COLLABORATION PROTOCOL (SSOT GUIDELINES)

All four team members and their respective AI tools (Cursor, Claude Code, Antigravity, Copilot) must follow this protocol:

Universal AI Rules of Engagement

- **Never Rename Schema:** Model names (`dustwatch.site` , `reading` , `job`) and fields are locked.
- **Odoo 19 Syntax:** Use `list` (not tree), modern `@api.depends` , and avoid deprecated syntax.
- **Git First:** Every feature branch must branch from `main` and rebase before merging.

Git Branching Strategy

- `main` : Production-ready, validated code only.
- `feat/odoo-models` : Backend ORM and calculation methods.
- `feat/odoo-views` : XML views, actions, and menus.
- `feat/canva-presentation` : Pitch deck assets and demo scripts.

12. PRE-PACKAGED AI CONTEXT PROMPTS FOR SUB-TEAMS

Prompt for Pair 1 AI Assistants (The Frame / Developers)

"You are working on DustWatch, an Odoo 19 module. You must strictly follow SPECIFICATION.md in the repository root. Use models 'dustwatch.site', 'dustwatch.reading', 'dustwatch.job', and 'res.partner' extension. Ensure all compute methods use @api.depends and store=True where defined. Do not alter field names."

Prompt for Pair 2 AI Assistants (Presentation & Canva)

"You are building the Canva pitch deck and business model for DustWatch. The core soundbite is 'Today owners clean by the calendar. DustWatch lets them clean by the numbers.' Base all calculations on 500 kWp Grand Hyatt roof: AED 8,500/yr savings, AED 425k for 50 roofs, zero wasted wash water. Follow the 12-slide storyboard in SPECIFICATION.md."

13. 48-HOUR SPRINT ROADMAP & ACCEPTANCE CRITERIA

Phase	Target Milestone	Definition of Done (DoD)
Hour 0–12	Architecture & Skeleton	All models, views, wizards, security groups, cron, and sample data committed to Git.
Hour 12–24	Ingestion & View Polish	Module installs cleanly in Odoo 19; CSV wizard ingests 90 days; color decorators verified.
Hour 24–36	Canva Deck & ROI Model	12-slide pitch deck complete; Grand Hyatt sandstorm demo scenario rehearsed.
Hour 36–48	Final Polish & Dry Run	5-minute timed presentation rehearsal; judge Q&A defense drill; release tagged on Git.